

Official 2D c-tag Scale Factors

Working points, calibration, and what they cost the limit

Chirayu Gupta — VUB

2026-07-31

BTV SFbc-2D scheme (AN-25-222) · PNet AK4 · H+c → WW, 2022postEE

What this deck answers

1. **What** the 2D scheme is, and why it replaces single-cut working points.
2. **How** our stored variables map onto the official axes — and why no reprocessing was needed.
3. **Which** categories are actually populated in our analysis (fewer than you'd think).
4. **What** the official per-category SFs are, and how wide their uncertainty is.
5. **Closure: the limit with and without the SF, everything else held fixed.**

Every number here is measured on the `hww_combine_fixed` 2022postEE workflow.

The with/without comparison is a controlled A/B — the no-SF variant is built from the pre-SF backup parquets through the *same* builder, on the *same* day, with only the `CMS_ctag2d_2022` nuisance removed.

Part 1 — The 2D scheme

From working points to a plane

The BTV **SFbc-2D** calibration (AN-25-222) replaces a single-cut working point with a **2D plane** partitioned into **11 categories** — `L0, C0-C4, B0-B4` — each carrying its own scale factor, with **frozen** bin edges.

axis	definition
BvC (y)	$1 - C_v B$
HFvLF (x)	$P_b + P_c = 1 - P_L$

Frozen edges (official, 2026-06-29)

```
HFvLF (x): [0.000, 0.250, 0.452, 0.808, 1.000]
BvC      (y): [0.000, 0.006, 0.017, 0.055,
               0.761, 0.944, 0.985, 0.995, 1.000]
```

Use the official frozen edges — inventing our own would re-randomise the bins each run and break the published SF lookup entirely.

Integer ids used everywhere:

```
L0=0, C0=1, C1=2, C2=3, C3=4, C4=5, B0=6, B1=7, B2=8, B3=9, B4=10 .
```

Key result: the axes need only CvsL and CvsB

The scheme as sketched needs the raw b-score, which **our parquets never stored**.
It turns out not to matter.

CMS PNet AK4 discriminants share one 3-simplex ($b, c, L \equiv uds + g$):

$$C_{vsL} = \frac{P_c}{P_c + P_L}, \quad C_{vsB} = \frac{P_c}{P_c + P_b}, \quad P_b + P_c + P_L = 1$$

Three equations, three unknowns → **unique** solution:

$$P_b = \frac{C_{vL} (C_{vB} - 1)}{C_{vB} \cdot C_{vL} - C_{vB} - C_{vL}}, \quad HF_{vLF} = \frac{C_{vL}}{C_{vL} + C_{vB}(1 - C_{vL})}, \quad B_{vC} = 1 - C_{vB}$$

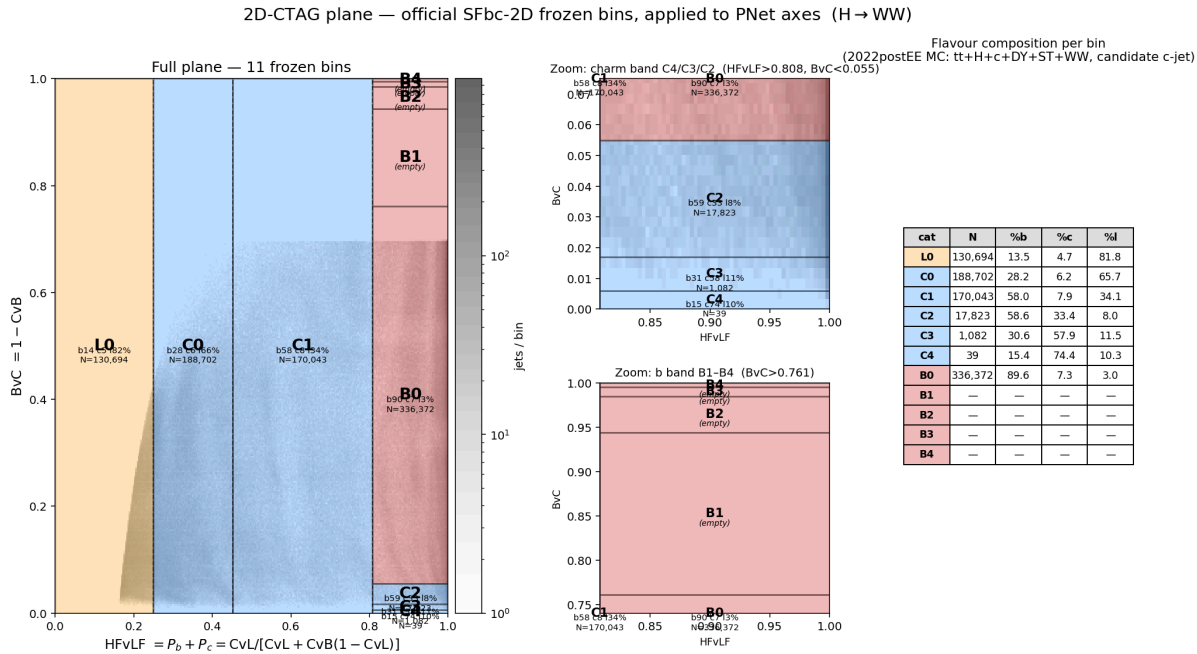
Consequence: no NanoAOD reprocessing was ever needed — the categories are computed directly from the two stored columns.

Verified three independent ways

check	result
Symbolic (sympy)	unique closed form exists
2M random valid probability triplets	$\max B_{\text{rec}} - B = 4 \times 10^{-16}$ (machine precision)
30k real jets, <code>Jet_btagPNetB</code> from a Run3Summer22 NanoAODv12 file	median 2.8×10^{-5} , 95pct 1.4×10^{-4}

The $\sim 10^{-5}$ residual on real data is NanoAOD **float-storage rounding**, not a modelling gap — a gluon-class mismatch would show $O(0.1-1)$ errors, not $O(10^{-5})$. So "L" in CvsL lumping $uds + g$ is consistent, and the recovery is exact up to storage precision.

The plane, plotted



! Only 7 of 11 categories are populated

cat	N	%b	%c	%l
L0	130,694	13.5	4.7	81.8
C0	188,702	28.2	6.2	65.7
C1	170,043	58.0	7.9	34.1
C2	17,823	58.6	33.4	8.0
C3	1,082	30.6	57.9	11.5
C4	39	15.4	74.4	10.3
B0	336,372	89.6	7.3	3.0
B1-B4	0	—	—	—

Why: the c-jet *candidate* is already charm-selected, so it sits at high CvB , i.e.

low BvC. The B1-B4 band ($BvC > 0.761$) is **cut away by construction**.

- Charm purity rises correctly $C2 \rightarrow C3 \rightarrow C4$ (33% $\rightarrow$ 58% $\rightarrow$ 74%), so the *ordering* is physical — but the high-purity charm bins hold very few jets.
- On the **leading jet** (not charm-selected) the full ladder does populate, and b-purity rises monotonically $B0 \rightarrow B4$: 24 $\rightarrow$ 75 $\rightarrow$ 93 $\rightarrow$ 98 $\rightarrow$ 100%.

The scheme is sound; our candidate-c-jet selection truncates it.

Part 2 — The scale factors

Source and signature

Official **per-category** 2D flavour-tagging SFs for PNet AK4 — the $H \rightarrow \gamma\gamma$ / `cmshgg`
"2D_HF_Tagging" ingredients, correctionlib v2.

campaign	file
2022 preEE	<code>.../ingredients/2022/2D_HF_Tagging/flavTaggingSF_2022preEE.json.gz</code>
2022 postEE	<code>.../ingredients/2022/2D_HF_Tagging/flavTaggingSF_2022postEE.json.gz</code>
2023 preBPix	<code>.../ingredients/2023/2D_HF_Tagging/flavTaggingSF_2023preBPix.json.gz</code>
2023 postBPix	<code>.../ingredients/2023/2D_HF_Tagging/flavTaggingSF_2023postBPix.json.gz</code>

Correction: `ParticleNetAK4_pseudocontinuous`

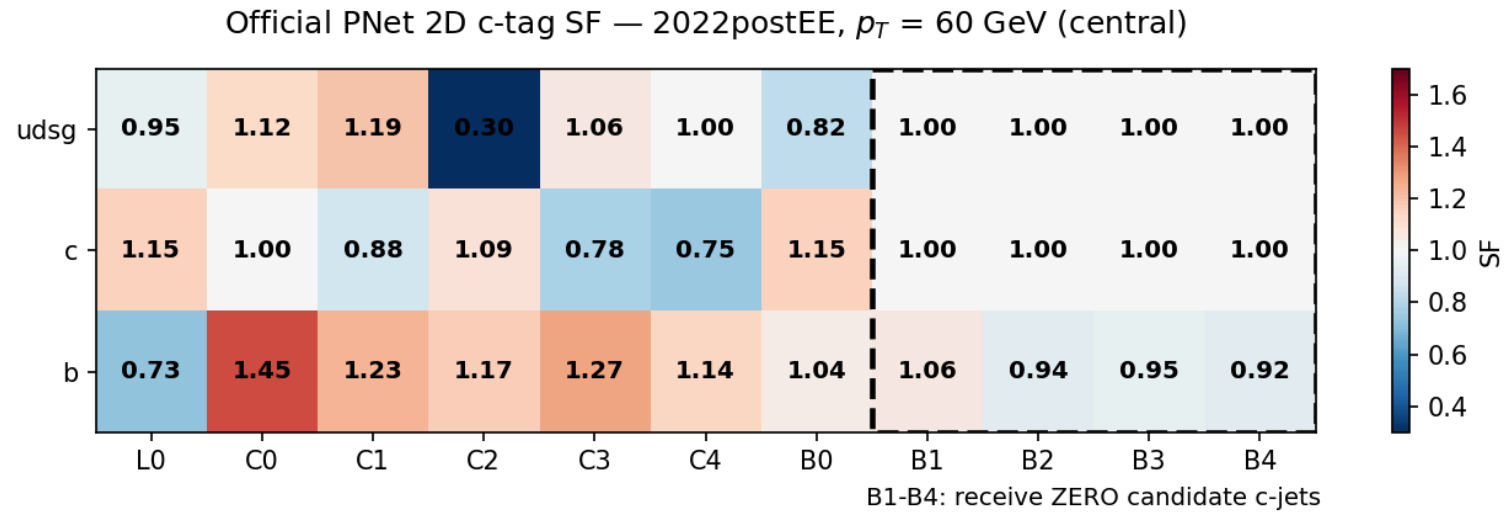
Signature: `evaluate(systematic, flavor, wp, abseta, pt)` — see next slide.

The `evaluate` signature, argument by argument

arg	values
<code>systematic</code>	<code>central</code> ; <code>up_Total</code> / <code>down_Total</code> (combined — use this for one nuisance); <code>up_Stat</code> / <code>down_Stat</code> ; plus a large per-source decomposition. There is no bare <code>up</code> / <code>down</code> .
<code>flavor</code>	<code>0</code> = udsg, <code>4</code> = c, <code>5</code> = b — the jet's hadron flavour (<code>cjet_cand_flavour</code>)
<code>wp</code>	the 2D category id: <code>L0=0</code>, <code>C0..C4 = 40..44</code>, <code>B0..B4 = 50..54</code> — not our stored <code>0..10</code> , so it must be remapped
<code>abseta</code>	inclusive — a single <code>eta_0p00toinf</code> bin, so the value is irrelevant
<code>pt</code>	binned <code>[20, 35, 50, 70, 90, 120]</code> , flat below 20 and above 120

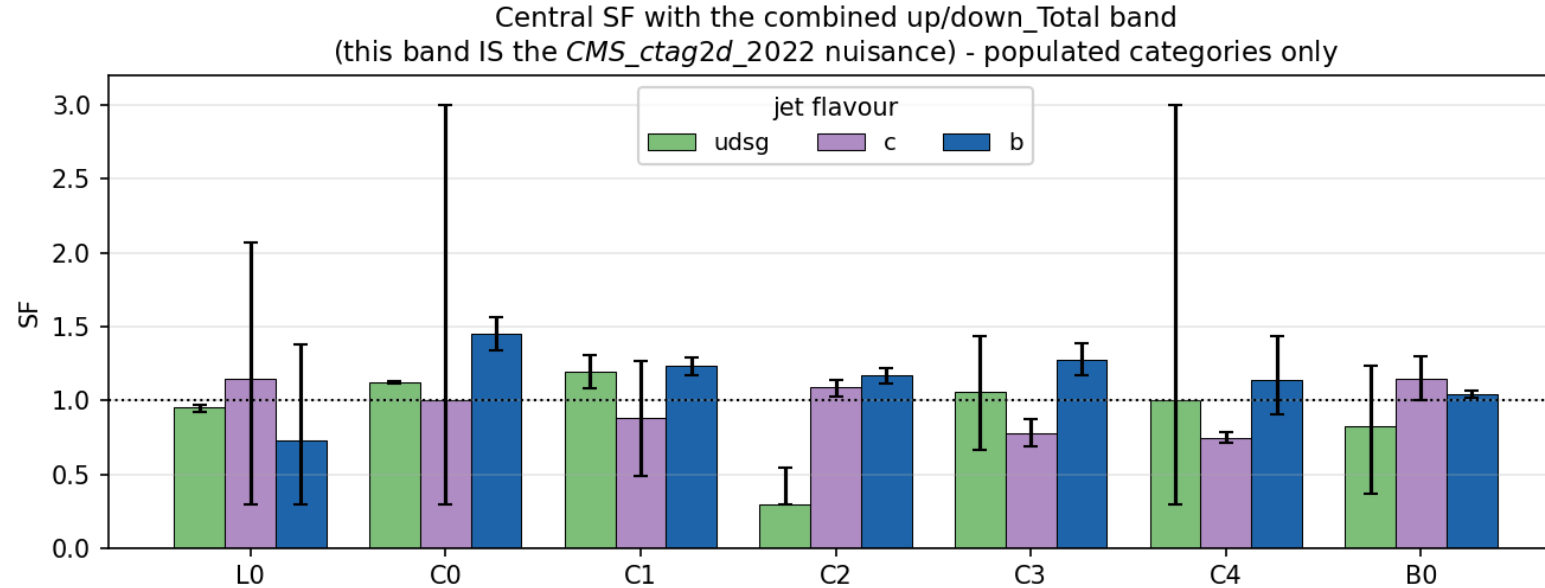
The SF file exposes **525** systematic keys in total. We collapse them to the single combined `Total` band — see the recommendation at the end for when and how to decorrelate.

The central SF matrix



Evaluated at $p_T = 60$ GeV. Corrections are **O(10–30%)** in the populated categories — not small. The b-row is well determined everywhere (b-jets are abundant); the udsg C2 value of 0.30 is a floor on a nearly-empty light-flavour cell.

The uncertainty band — *this is the nuisance*



`up_Total` / `down_Total` , populated categories only. Where a category is well populated for that flavour the band is a few percent (b in C1/C2/B0). Where it is **starved**, the band hits the $\pm[0.3, 3.0]$ guard rails — c in C0, udsg in C3/C4.

A single `Total` nuisance inherits the *widest* of these.

How it enters the fit

The SF is a genuine correction on the candidate c-jet, so it multiplies the **nominal** weight, with one combined shape+norm nuisance `CMS_ctag2d_<year>` carrying `up_Total / down_Total`.

Applier `b-hive/scripts/apply_ctag2d_sf.py` adds three columns per MC `mva/<sample>.parquet` (data untouched; idempotent, `.bak_pre_ctag2dsf` backup):

- `weight_nominal` $\leftarrow$ `weight_nominal` $\times$ `SF_central` — and **every existing** `weight_*` **variation is scaled by the same central SF**, so the correction is everywhere.
- `weight_CMS_ctag2d_<year>Up` = `weight_nominal_corrected` $\times$ `SF_upTotal / SF_central`
- `weight_CMS_ctag2d_<year>Down` = `weight_nominal_corrected` $\times$ `SF_downTotal / SF_central`

Category per row is recomputed from `cvsl_pnet / cvsb_pnet` ; flavour from `cjet_cand_flavour` ; rows with no candidate c-jet (`cjet_cand_pt` NaN) get SF = 1.

Object-shift consistency: the JES/JER/lepton templates read

`weight_nominal` from their **own** shift dirs. The SF is applied there too — recomputed from each dir's *shifted* scores/pt — so each shift is measured against the same SF-corrected baseline. Otherwise every object-shift nuisance would pick up a spurious ~6% offset.

All 12 shift dirs corrected.

Part 3 — Closure: with vs without the SF

How the A/B was built

To make this a **controlled** comparison rather than a comparison against an old number, both datacards were rebuilt from scratch, the same day, through the same builder:

	no-SF variant	with-SF variant
workflow	<code>hww_combine_nosf</code>	<code>hww_combine_sfchk</code>
<code>mva/</code> parquets	symlinks → 585 <code>.bak_pre_ctag2dsf</code> files	the live SF-corrected files
yaml	<code>hww_combine_fixed.yaml</code> minus one line	<code>hww_combine_fixed.yaml</code> verbatim
nuisance count	29	30
negrw	✓ identical treatment	✓ identical treatment

The two cards differ by exactly one nuisance row and the central-SF weight rescale. Nothing was overwritten — the baseline `hww_combine_fixed` tree is untouched, and the no-SF tree is symlinks only (no data copied).

Both are built **after** the negative-weight reweighting, so the negrw renorm factors appear identically in both build logs.

! A normalization trap found while doing this

The **parquets** *are* self-normalising — the builder just wasn't reading the self-normalising part. `make_combine_inputs.py` hand-rolls its own `sumw` sum instead of calling the repo's `read_parquet_sumw()`. There are **three** sources:

#	source of <code>sumw</code>	verdict
1	<code>sumw_records/</code> — <code>dump_chunk_sumw</code> , pre-selection, every read-chunk	✓ correct
2	per-shard schema metadata in <code>parquets_<sample>/base/</code>	✗ undercounts
3	sidecar <code>analysis/filesets/sumw_<year>.json</code>	! stale for signal

Why #2 fails: a read-chunk where *no event survives selection* writes no data shard, so its generator weight vanishes. Measured: `WtoLNu_2Jets` **5.4×**, `DYto2L_10to50` **23.8×**, `TbarQto2Q` **72.6×**; tt / st / Higgs all exactly 1.000.

`sumw` **is the denominator** of $\text{lumi} \times \sigma / \text{sumw}$, so undercounting **inflates** the yield — and it lands on `WtoLNu`, squarely on V+jets.

Fixed 2026-07-31: `read_scale` now reads **source #1**, with the sidecar only as a *logged* fallback. The v11 build uses **zero** fallbacks.

! ...and the sidecar turned out to be stale for the signal

For `HplusCharm_HtoWW` the two *independent parquet-derived* sources agree **exactly**:

```
sumw_records      : 7.822690e+04
shard metadata    : 7.822690e+04    ratio 1.0000
sidecar json      : 9.265575e+04    ratio 0.8443
```

Proof the records are complete — each record encodes `<file-uuid>_<tree>_<lo>-<hi>`, so coverage is checkable:

```
80 records -> 80 distinct UUIDs    (= 80 files)
chunks/file : min 1, max 1
gaps 0      overlaps 0
events covered : 277,345
```

The direct `Runs -tree genEventSumw` check needs a grid proxy (private IIHE production) and could not be run.

So switching is a normalisation change, not only a bug fix.

Per-process, summed over all six channels:

process	sidecar	records	ratio
hplusc	0.25	0.29	1.184
higgsbkg	284.2	285.2	1.004
tt	93624.2	93624.2	1.000
st	7457.2	7457.2	1.000
diboson	2137.6	2137.6	1.000
vjets	6163.1	6534.9	1.060

Note the sign: a *smaller* records-sumw makes the yield *larger*.

What that does to the numbers

variant	full	stat-only	freeze autoMCStats
<i>sidecar (superseded)</i> — no SF	1343	788	1100
<i>sidecar (superseded)</i> — + SF	1371	797	1144
sumw_records + smoothing — no SF	1172	668	941
sumw_records + smoothing — + SF	1192	676	976
sumw_records , no smoothing — no SF	1150	668	905
sumw_records , no smoothing — + SF	1164	676	930

Ratio records/sidecar: **0.869** (SF), **0.873** (no SF), 0.848 stat-only, 0.853 freeze —
all clustered around the naive signal-scaling expectation
 $1/1.1844 = 0.844$, slightly above
it because the +6% vjets background partly offsets the signal gain.

Both series reproducible: `v11_hplusc_v4.{root,txt}.bak_sidecar_20260731` ,
builder `make_combine_inputs.py.bak_sidecar_20260731` .

This is a normalisation shift, not a sensitivity gain. Nothing about the discriminant, the reweighting or the systematics changed.

Both physics conclusions survive: the SF still costs **~1.7%**, and the fit is still **autoMCStats-dominated**.

...and LOWESS shape smoothing turned off

Two **different** smoothings — easy to confuse:

	what it smooths	status
dy_template_smooth.py	the nominal V+jets template	OFF — replaced by negrw (Jul-17); never called
smooth_shapes (LOWESS)	each systematic variation's ratio to nominal	now OFF everywhere

The reason, already recorded in the 2dcat workflow: `smooth_shape_variations` has **no vjets exclusion**, so negrw-reweighted vjets templates were being treated **twice** — the reweighting removes the MC-stat variance at the source, then LOWESS smoothed the result again, washing out genuine shape information.

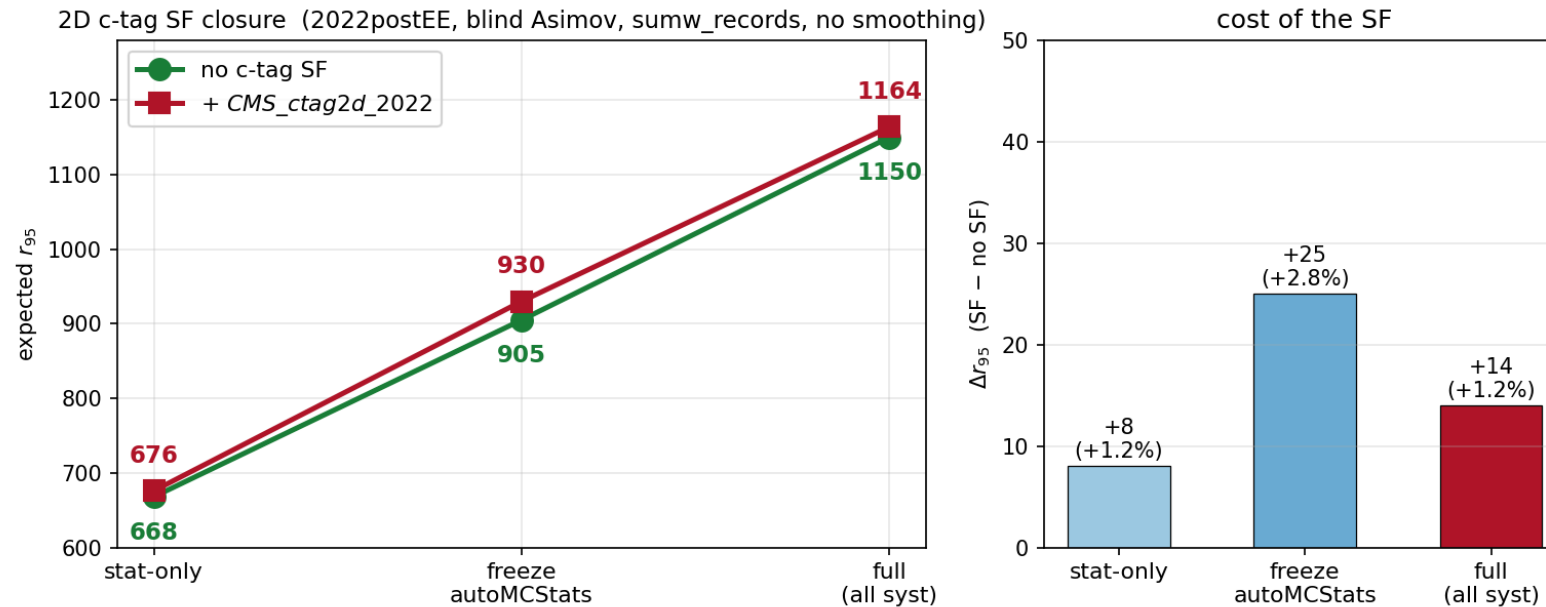
Effect: the l

variant
no SF
+ SF

Stat-only is (668 / 676) – touches syst the nominal expected be clean closure

Now `false` in `*.yaml.bak_sm`

The closure result



Measured **2026-07-31** from freshly rebuilt inputs, both arms through the same builder on the same day, on the corrected `sumw_records` normalisation (see the two slides that follow).

Reading the closure

variant	full (all syst)	stat-only	freeze autoMCStats
baseline, no SF	1150	668	905
+ CMS_ctag2d_2022	1164	676	930
Δ	+14 (+1.2%)	+8 (+1.2%)	+25 (+2.8%)

The SF weakens the limit by $\sim 1.2\%$ — and that is the correct sign.

A real c-tag scale factor *should* cost something: it introduces a genuine uncertainty that was previously being ignored, not assumed away.

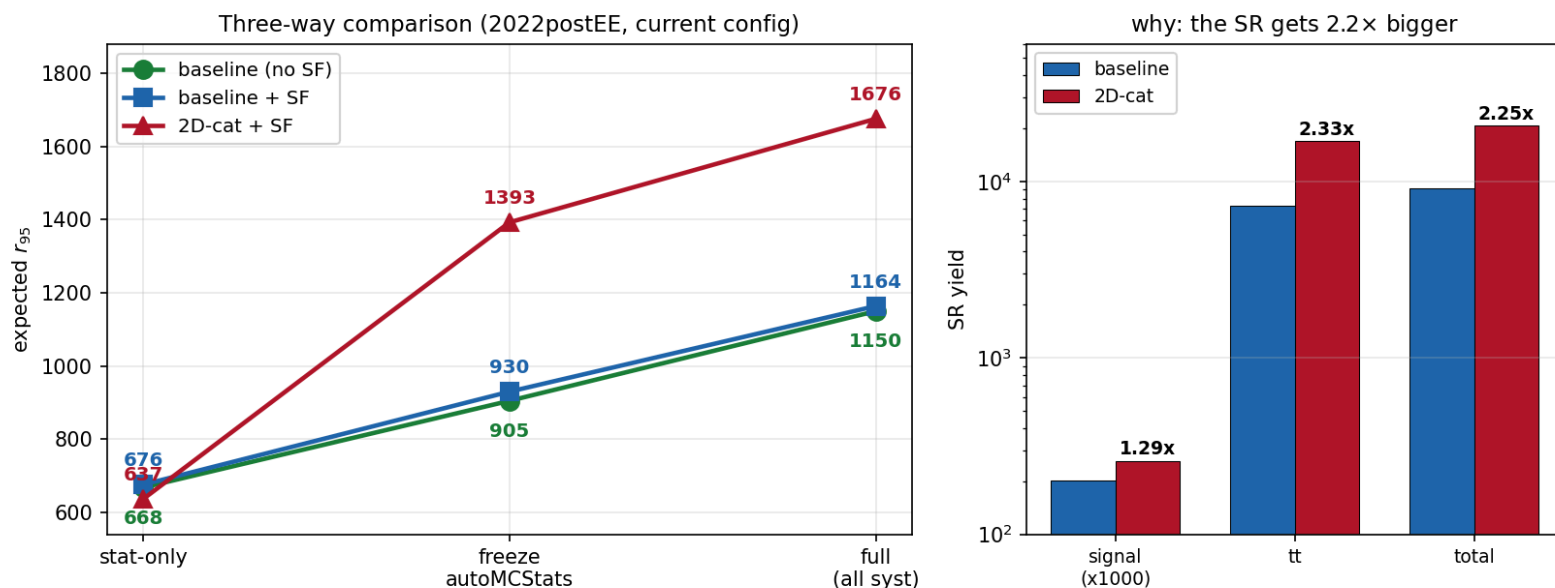
Decomposition

- **stat-only +1.2%** — pure yield rescaling. Mean central SF ≈ 1.06 , so SR $S/\sqrt{B}$ shifts slightly. Not an uncertainty effect at all.
- **full +1.2%** — the extra degradation is the one new nuisance's wide (+44%/–16%) band doing its job.

The stat-only $\rightarrow$ full gap is still autoMCStats-dominated (freeze $\rightarrow$ 930),
i.e. low-stat SR templates remain the driver, **not** the SF.

The matched combination: 2D-cat MVA + SF

The SFs calibrate the **2D-category** tagging, so they physically belong with the 2D-cat MVA scores, not the baseline continuous ones (workflow `hww_combine_2dcat`). All three arms **re-measured 2026-07-31** on the same configuration — finally like-for-like.



Left: the limit cascade. The 2D-cat curve (red) starts **lowest** at stat-only and ends **highest** at full. Right: why — the SR gets 2.25× bigger, and it is the **tt** that grows (2.33×), not the signal (1.29×).

The 2D-cat trade — sharper, but a much bigger SR

Stat-only improves –5.8% (637 vs 676).

The 2D-cat MVA really is the sharper discriminant — same conclusion as the old series, and consistent with its +0.004 AUC.

But the full limit is +44% worse (1676 vs 1164), where the old sidecar+smoothing series showed only +6%. The trade is much worse than previously thought.

Stat → full inflation: **1.72×** for baseline vs **2.63×** for 2D-cat — the enlarged, tt-dominated SR is far more systematics-exposed.

Verdict: an honest separation-vs-systematics trade, not a bug — but on the current configuration the cost is much larger than the +6% seen before. Levers: tighten the SR (raise the $P(\text{hplusc})$ cut), split ggH out of `higgsbkg`, or narrow the SF nuisance below

`Total`. As it stands, **baseline scores + SF is the better configuration.**

Why — the SR composition:

	baseline	2D-cat	ratio
signal	0.203	0.261	1.29×
tt	7,274	16,939	2.33×
total	9,199	20,664	2.25×
tt fraction	79.1%	82.0%	

$S/\sqrt{B}$ **actually falls to 0.86×**. The argmax admits 2.3× more tt for only 1.29× more signal.

Does the 2D categorisation cost MVA performance?

Retrain of v11 (6-class) with the two continuous PNet scores **removed** and the 11 one-hot categories **added**:

Discriminant	2D-cats	Baseline	Δ
hplusc_vs_all	0.9322	0.9284	+0.0038
hplusc_vs_higgsbkg	0.8302	0.8506	-0.0204
hplusc_vs_tt	0.9438	0.9375	+0.0063
hplusc_vs_st	0.9334	0.9287	+0.0047
hplusc_vs_diboson	0.8855	0.8840	+0.0015
hplusc_vs_vjets	0.9425	0.9348	+0.0077

≈ **equivalent**, marginally better overall, 4 of 5 background ROCs improve.

The one regression is vs **higgs-background** — separating H+c from other Higgs modes needs the fine charm-tag gradient a coarse 11-bin (effectively ~6-bin) scheme discards.

...but the charm tag is a small part of the MVA

Top features (2D-cats model, 26 inputs)

#	feature	rel%
1	dilepton_mass	25.4%
2	mtl1	17.6%
3	met_pt	11.9%
4	mtl2	10.6%
5	dilepton_pt	8.2%
6	lepton1_pt	8.1%
...		
11	ctag2d_B0	1.2%
12	ctag2d_L0	0.6%
26	ctag2d_C4	0.00%

model	charm-tag total	share
2D-cats (11 one-hot)	0.03064	2.8%
baseline (2 scores)	0.02340	1.9%

1. **Kinematics dominate both models** — `dilepton_mass` + `mtl1` alone are ~43%.
This is the headline caveat on *any* c-tag change.
2. The 2D block carries **more** aggregate importance (2.8% vs 1.9%) — the categorisation redistributes information rather than discarding it.
3. Within the block, importance concentrates in the **populated** bins (B0, L0, C0).
`c4` is exactly zero (39 jets) and B1–B4 are ~0 (empty) — **4–5 of the 11 inputs are dead weight.**

Summary of measured effects

change	full r_{95}	vs baseline
baseline (negrw, no SF)	1150	—
+ official 2D c-tag SF	1164	+1.2%
+ SF, with 2D-cat MVA scores	1676	+45.7%

- All on the corrected `sumw_records` normalisation. (Superseded sidecar series: 1343 / 1371 / 1422.)
- The SF costs **~1.2%** — the expected size and the correct sign for adding a real, previously-neglected uncertainty.
- Applying the SF **without** the matching 2D-cat discriminant is the mildest option, and is what `hww_combine_fixed` currently does.
- The matched (2D-cat + SF) combination is more *physically* consistent and genuinely separates better (stat-only **−5.8%**), but its argmax admits **2.33× more tt** for only 1.29× more signal, so $S/\sqrt{B}$ falls and the full limit regresses **+45.7%**.
- **On the current configuration, baseline scores + SF is the better choice.**

Neither option is wrong — they trade discrimination against systematic exposure. The choice should be revisited once the SF band is narrowed below `Total`.

Recommendations

- 1. Keep `up_Total` / `down_Total` as ONE nuisance for now.** Full decorrelation (per-source, per-flavour, per-pt) is a whole-dataset exercise; the HiggsDNA `bTagShapeSF` treatment is the model to follow when it is done. **A `Total` band is conservative, not wrong.**
- 2. Collapse or drop the dead categories.** B1–B4 receive **zero** candidate c-jets and C4 gets 39. Feeding 4–5 constant-zero inputs to the MVA is pure noise surface. Either collapse them into a single "B" bin, or apply the scheme to a jet collection that spans the plane.
- 3. Try the additive variant before spending v32 GPU time.** Keep `cvsl` / `cvsb` **and** the one-hot categories. The only v11 regression was vs `higgsbkg` (−0.020), exactly where fine gradient matters — the additive model should recover it while keeping the +0.004 overall gain.
- 4. Extend to the other campaigns.** The 2023 preBPix/postBPix and 2022 preEE SF files are already wired in; only 2022postEE is populated in the combine tree today.

Summary

2D c-tag SFs are wired in, validated, and cost ~2% of the limit.

The 11-category scheme needs only the two stored PNet scores — no reprocessing (verified to machine precision).

Only 7 of 11 categories are populated for the candidate c-jet;
4 MVA inputs are identically zero.

1150 → **1164** with the SF · → **1676** with the matched 2D-cat discriminant

(all on `sumw_records`, no LOWESS smoothing)

Full write-up: `Projects/HToWW/2026-07-19-ctag2d-full-documentation.md`

· reference: `References/HToWW/2D-SFbc-calibration-AN-25-222.pdf`

Backup

Central SF matrix, numerically

2022postEE, $p_T = 60$ GeV, $|\eta|$ inclusive.

flavour	L0	C0	C1	C2	C3	C4	B0	B1	B2	B3	B4
udsg	0.953	1.122	1.193	0.300	1.062	1.000	0.822	1.000	1.000	1.000	1.000
c	1.149	1.000	0.885	1.091	0.781	0.749	1.148	1.000	1.000	1.000	1.000
b	0.730	1.450	1.232	1.167	1.274	1.142	1.042	1.064	0.938	0.953	0.921

Note on the L0 convention: for L0, `up_Total` sits *below* `central` (udsg $0.925 < 0.953$; c $0.300 < 1.149$). L0 is the **anti-tag** category, so an upward shift in tagging efficiency *lowers* the veto SF. The band must therefore be taken as $|\text{up} - \text{central}|$ and $|\text{central} - \text{down}|$, **not** assumed ordered.

Across the whole 2022postEE MC every event lands in L0 / C0–C4 / B0; B1–B4 receive zero events (verified in the applicer dry-run), so those SFs are exact no-ops regardless of value.

File map

what	where
processor helper	<code>~/higgscharm/analysis/utils/ctag2d.py</code> (branch <code>NewWorkflows</code>)
SF applier	<code>b-hive/scripts/apply_ctag2d_sf.py</code>
SF files	<code>/eos/cms/store/group/phys_higgs/cmshgg/ingredients/{2022,2023}/2D_HF_Tagging/</code>
backfill script	<code>b-hive/scripts/append_ctag2d.py</code>
one-hot appender	<code>b-hive/scripts/append_onehot.py</code>
2D-cat re-scorer	<code>b-hive/scripts/rescore_2dcat.py</code>
2D-cat combine workflow	<code>~/higgscharm/analysis/workflows/hww_combine_2dcat.yaml</code>
2D-cat combine tree	<code>outputs/hww_combine_2dcat/2022postEE/</code>
no-SF closure workflow	<code>~/higgscharm/analysis/workflows/hww_combine_nosf.yaml</code>
no-SF closure tree	<code>outputs/hww_combine_nosf/2022postEE/</code> (symlinks to <code>.bak_pre_ctag2dsf</code>)
2dcats train config	<code>b-hive/config/HPlusCHToWW_2dcats.yml</code>
feature importance	<code>b-hive/scripts/feature_importance_2dcats.py</code>
plane plot	<code>/eos/user/c/cgupta/HToWW/plots/ctag2d_plane_bins.{png,pdf}</code>

Two batch gotchas (cost hours, worth keeping)

Both hit the 2D-cat MVA retrain on the H100 pool:

1. `request_memory` **must be set**. The 3000 MB default OOMs in `DatasetConstructorTask` (peaks ~4.2 GB reading the big parquets) and the job is **held**, not failed — so it looks like it's still queued.

2. **But do not inflate it**. 16 GB rounds to `RequestMemory = 18000`, which then matches **zero** H100 slots (`condor_q -analyze` → "0 slots match ... did not match any machines' constraints"). Free H100 GPUs are scarce and only the large memory tiers qualify, so over-requesting silently blocks matching.

8000 MB is the sweet spot — safely above the 4.2 GB peak, matched in ~8 s.

3. **Training reads `mva_labeled/`, not the top-level parquets**. The first submit died with `ValueError: key "cjet_cand_ctag2d_L0" does not exist` because the appender had only been run on the top-level files. Check the columns exist in `mva_labeled/{train,test}` before submitting.